

PROJETO: CONEXÕES SEGURAS

Documentação Técnica — Prova de Conceito (PoC) • Parte 1/3

Resumo das Entregas Estruturais

Entrega Obrigatória	Descrição do Elemento	Arquivo Correspondente no Repositório
Relatório Técnico	Documentação teórica e analítica das práticas (máx. 5 páginas).	relatorio.pdf (Raiz do repositório)
Implementação Prática	Scripts, códigos-fonte e evidências de execução da PoC por módulo.	Pastas de modulo1/ a modulo4/
GitHub Pages	Documentação online integrada com links operacionais.	README.md de suporte

Módulo 1 — Sniffing e Interceptação (Camadas 2 e 3)

Ferramentas utilizadas: Wireshark 4.2 / tcpdump 4.99

Em operação padrão, a Placa de Rede (NIC - Network Interface Card) analisa os quadros de dados que chegam ao barramento físico e aplica uma filtragem via hardware baseada no Endereço MAC de destino. Se o destino não for o endereço da própria interface (ou um endereço de broadcast/multicast), o controlador descarta os dados para poupar processamento do sistema operacional.

O **Modo Promíscuo** consiste na desativação explícita dessa filtragem de hardware. Por meio de uma chamada de controle do sistema (como o `ioctl` com a flag `IFF_PROMISC` em sistemas Linux), o driver instrui a NIC a repassar 100% dos fluxos de dados capturados diretamente para a pilha de protocolos do kernel (TCP/IP), permitindo o monitoramento de tráfego de terceiros que transita pelo mesmo segmento de rede.

PASSO A PASSO DA EXECUÇÃO

```
# 1. Ativação do modo promíscuo na interface eth0
sudo ip link set eth0 promisc on
ip link show eth0           # Confirmação visual da flag PROMISC no cabeçalho

# 2. Captura analítica de pacotes via tcpdump limitando a 100 registros
sudo tcpdump -i eth0 -c 100 -w modulo1-sniffing/evidencias/captura.pcap

# 3. Filtros aplicados no Wireshark para isolar tráfego HTTP exposto
http                        # Isola requisições de aplicação em texto claro
ip.addr == 192.168.1.5      # Filtro focado no endereço IP do host alvo

# 4. Desativação do modo promíscuo após encerramento da PoC
sudo ip link set eth0 promisc off
```

RESULTADO E EXPLICAÇÃO TÉCNICA

O Wireshark passou a exibir os quadros trafegados na rede independentemente do destino final. Em redes legadas com hubs (Camada 1), o broadcast é físico e absoluto, facilitando a captura total. Em cenários modernos governados por Switches (Camada 2), a tabela CAM (Content Addressable Memory) segmenta o

tráfego por porta. Para contornar essa restrição na PoC e validar os riscos, o modo promíscuo foi aliado à técnica de *ARP Poisoning*, forçando o redirecionamento dos pacotes para a estação de monitoramento.

O principal achado técnico foi a interceptação de pacotes HTTP (Camada 7): as requisições expõem parâmetros, caminhos e credenciais em texto claro, comprovando que a infraestrutura local é vulnerável a ataques de interceptação caso não adote criptografia de ponta a ponta.

Evidências do Repositório: [modulo1-sniffing/evidencias/01_wireshark_captura.png](#) | [02_promisc_ativo.png](#) | [captura.pcap](#)

Módulo 2 — Criptografia e Handshake (A Base do TLS)

Ferramentas utilizadas: OpenSSL 3.x / Python ssl 3.11 / Wireshark

O protocolo HTTPS não emprega chaves assimétricas (como RSA ou ECC) para a cifragem de todo o tráfego devido ao alto custo computacional. Algoritmos assimétricos são baseados em exponenciações modulares complexas com números primos extensos, operando de forma exponencialmente mais lenta que a criptografia simétrica. Sob carga corporativa massiva, o uso exclusivo de criptografia assimétrica geraria gargalos severos de processamento nas CPUs dos servidores.

A arquitetura adota a **Criptografia Híbrida**: algoritmos assimétricos entram em ação unicamente no início da conexão (Handshake) para autenticar o servidor e realizar a troca segura de um segredo compartilhado. Assim que esse segredo é estabelecido, ambas as pontas derivam chaves simétricas (como o padrão AES-256-GCM), que realizam a cifragem do fluxo massivo de dados em alta velocidade com consumo mínimo de hardware.

COMPARATIVO DE DESEMPENHO E APLICAÇÃO

Algoritmo Base	Tipo Criptográfico	Desempenho Relativo	Função Específica no Fluxo TLS 1.3
RSA-2048 / ECC	Assimétrico	Lento (~5.000 ops/seg)	Autenticação inicial e validação do Certificado Digital.
ECDHE (Curve25519)	Diffie-Hellman Efêmero	Intermediário	Acordo de chaves com garantia de PFS (Perfect Forward Secrecy).
AES-256-GCM	Simétrico	Ultra veloz (>1M ops/seg)	Cifragem volumosa de dados e validação de integridade (AEAD).

PASSO A PASSO DA EXECUÇÃO

```
# 1. Geração de um par de chaves e certificado autoassinado (X.509)
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -days 365 -nodes -subj
"/CN=localhost/O=ConexoesSeguras/C=BR"

# 2. Execução de script Python para validação e inspeção do Handshake TLS real
import ssl, socket
ctx = ssl.create_default_context()
with socket.create_connection(('www.google.com', 443)) as s:
    with ctx.wrap_socket(s, server_hostname='www.google.com') as ss:
        print(f"Protocolo: {ss.version()} | Cipher Suite: {ss.cipher()}")
# Saída homologada: TLSv1.3 | ('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256)
```

RESULTADO E EXPLICAÇÃO TÉCNICA

A PoC validou a conformidade com o padrão TLS 1.3, reduzindo o handshake para apenas 1 RTT (Round Trip Time). Na fase inicial assimétrica, através do protocolo ECDHE, o cliente e o servidor geraram suas chaves efêmeras e derivaram de forma independente o segredo de sessão (Master Secret) sem que este

transitasse pela rede física. Isso assegura a propriedade de **Perfect Forward Secrecy (PFS)**: mesmo que a chave privada de longo prazo do servidor venha a ser vazada no futuro, as capturas de rede passadas não poderão ser descriptografadas, pois as chaves de sessão foram voláteis e exclusivas daquele handshake.

Evidências do Repositório: [modulo2-tls/evidencias/01_handshake_wireshark.png](#) | [02_cipher_suite.png](#) | [03_tls_output.txt](#)

Módulo 3 — Segurança na Camada de Aplicação (Desenvolvimento)

Ferramentas utilizadas: Python Flask 3.x / Burp Suite / Browser DevTools

O protocolo HTTP opera em modo *stateless* (sem estado), o que impede o servidor de associar nativamente requisições subsequentes ao mesmo usuário. Para contornar essa característica, após a validação bem-sucedida das credenciais de acesso, a aplicação web gera um **SessionID** — um identificador alfa-numérico único de alta entropia. O servidor armazena esse token em memória (ou banco de dados) e o injeta na resposta HTTP através do cabeçalho Set-Cookie. O navegador armazena o cookie e o retransmite de forma automática a cada nova requisição direcionada àquele domínio, autenticando as ações do usuário.

MAPEAMENTO DE VULNERABILIDADES E MITIGAÇÕES EM COOKIES

Vetor de Ataque	Mecanismo do Risco	Mitigação Implementada via Código
Session Hijacking	Interceptação do SessionID trafegando em texto claro na rede.	Ativação da flag Secure (força transmissão restrita ao HTTPS).
Cross-Site Scripting (XSS)	Scripts maliciosos injetados capturam o token via <code>document.cookie</code> .	Ativação da flag HttpOnly (bloqueia leitura via JavaScript).
Cross-Site Request Forgery	Requisições forjadas de domínios terceiros anexam o cookie automaticamente.	Configuração da flag SameSite=Strict (restringe ao contexto de origem).

PASSO A PASSO DA EXECUÇÃO

```
# Fragmento crítico do backend homologado em Flask (app.py)
from flask import Flask, request, jsonify
import secrets
app = Flask(__name__)
sessoes = {}

@app.route('/login', methods=['POST'])
def login():
    # Validação lógica de credenciais simulada
    sid = secrets.token_urlsafe(32) # Geração via gerador criptográfico seguro
    sessoes[sid] = {'user': 'admin'}
    resp = jsonify({'status': 'Autenticado com sucesso'})
    resp.set_cookie('sessionid', sid,
                    httponly=True, # Mitigação XSS
                    secure=True,    # Mitigação Sniffing
                    samesite='Strict') # Mitigação CSRF

    return resp
```

RESULTADO E EXPLICAÇÃO TÉCNICA

Durante os testes com ferramentas de inspeção (DevTools), verificou-se que qualquer tentativa de ler o valor do identificador via console utilizando comandos JavaScript (`document.cookie`) retornou vazia, comprovando a eficácia da proteção HttpOnly. Quando a aplicação foi forçada a trafegar em portas HTTP normais, o Wireshark capturou o cabeçalho Cookie: `sessionid=...` de forma exposta, permitindo o sequestro imediato de sessão por replay de pacotes. A combinação obrigatória de transporte TLS com

restrições explícitas de escopo nos cookies provou ser indispensável para neutralizar ataques na camada de aplicação.

Evidências do Repositório: [modulo3-sessao/evidencias/01_cookie_devtools.png](#) | [02_sessionid_http_exposto.png](#) | [03_https_cifrado.png](#)

Módulo 4 — VPN (Virtual Private Network)

Ferramentas utilizadas: WireGuard 1.0 / wg-quick / Wireshark

A VPN estabelece um canal seguro sobre uma infraestrutura de rede pública ou hostil através do conceito de **Encapsulamento IP-em-IP**. Os pacotes de dados originais gerados pela estação de trabalho (contendo cabeçalhos de rede internos com IPs privados) são integralmente criptografados e emoldurados como carga útil (*payload*) dentro de um novo pacote de rede externo. Este novo pacote recebe um cabeçalho de transporte público apontando o IP real do servidor VPN como destino, trafegando com total opacidade pelos nós da Internet.

PASSO A PASSO DA CONFIGURAÇÃO E CAPTURA

```
# 1. Instalação e geração de chaves criptográficas assimétricas no WireGuard
sudo apt update && sudo apt install wireguard -y
wg genkey | tee server_private.key | wg pubkey > server_public.key

# 2. Configuração do túnel no arquivo do Cliente (wg0-client.conf)
[Interface]
Address = 10.0.0.2/24
PrivateKey = <CHAVE_PRIVADA_CLIENTE>

[Peer]
PublicKey = <CHAVE_PUBLICA_SERVIDOR>
Endpoint = 198.51.100.1:51820
AllowedIPs = 0.0.0.0/0

# 3. Inicialização e monitoramento da interface virtual do túnel
sudo wg-quick up wg0-client.conf
sudo wg show # Validação do status de handshakes e tráfego
```

RESULTADO E EXPLICAÇÃO TÉCNICA

O funcionamento foi homologado por meio de requisições de checagem externa (`curl ifconfig.me`), as quais passaram a retornar o IP público do servidor da VPN e não mais o endereço físico local do cliente. No Wireshark, a análise comparativa entre as interfaces de rede gerou os seguintes resultados:

- **Interface Física (eth0):** Exibe exclusivamente pacotes sob o protocolo UDP direcionados à porta 51820. Os dados internos aparecem completamente ilegíveis, cifrados via algoritmo *ChaCha20-Poly1305*.
- **Interface Virtual (wg0):** Exibe o tráfego em estado descriptografado pelo driver do WireGuard, contendo os pacotes originais da aplicação (HTTP, DNS, ICMP).

Essa arquitetura resolve o problema exposto no Módulo 1: mesmo em um cenário de Modo Promísco e interceptação na rede física local, o atacante colhe apenas blocos binários de alta entropia sem utilidade prática, uma vez que não possui a chave privada correspondente.

Evidências do Repositório: [modulo4-vpn/evidencias/01_wg_show.png](#) | [02_ip_antes.png](#) | [03_ip_depois.png](#) | [04_wireshark_cifrado.png](#)

| Conclusão Estratégica

Os módulos desta primeira etapa se conectam formando uma estratégia robusta de **Defesa em Profundidade**. O Módulo 1 evidenciou a fragilidade intrínseca do meio físico. Em resposta, o Módulo 2 e o Módulo 3 agiram na proteção lógica das camadas superiores (Transporte e Aplicação), garantindo confidencialidade do canal e resiliência dos estados de autenticação. Por fim, o Módulo 4 estendeu a segurança para a Camada de Rede, criando um perímetro virtual blindado que protege todo o tráfego corporativo contra adversários locais e interceptações na rota externa.